WanderMate — Technical Report

Project: WanderMate — Social Mobile Travel Planner
Repository: github.com/arnav4124/WanderMate
Date: April 2026
Stack: React Native (Expo SDK 54) · Node.js/Express 5 · MongoDB Atlas · Firebase

---

Table of Contents

1. Executive Summary
2. Requirements & Scope
3. System Architecture
4. Design Decisions & ADRs
5. Design Patterns
6. Architectural Tactics
7. Data Models
8. Key Feature Implementations
9. External API Integration
10. Security Implementation
11. Offline-First Architecture
12. Real-Time Collaboration
13. Frontend State Management
14. Testing & Quality Assurance
15. Challenges & Lessons Learned
16. Conclusion

---

1. Executive Summary

WanderMate is a collaborative mobile travel-planning application that enables users to discover points of interest, construct day-by-day itineraries, optimise multi-stop routes, track shared budgets, and share trips through a social feed — all with real-time synchronisation across multiple collaborators.

The system is built as a full-stack application with a React Native mobile client (Expo managed workflow) communicating with a Node.js/Express REST API backed by MongoDB Atlas. Real-time features are powered by Firebase Realtime Database, authentication by Firebase Auth, and POI/routing capabilities by Google Places API and OpenRouteService (ORS).

The project demonstrates a production-grade architecture that balances feature richness, offline resilience, security, and maintainability across a complex multi-service system.

---

2. Requirements & Scope

2.1 Functional Requirements

ID	Requirement	Status
FR-01	User registration and login with email/password	✅ Implemented
FR-02	Trip creation with date ranges and auto-generated day structure	✅ Implemented

FR-03	POI search across 13+ categories with proximity sorting	✅ Implemented
FR-04	Add/remove/reorder stops within day itineraries	✅ Implemented
FR-05	Route optimisation using VROOM solver (ORS)	✅ Implemented
FR-06	Interactive map with colour-coded markers and route overlays	✅ Implemented
FR-07	Per-trip expense logging with category breakdown and per-person split	✅ Implemented
FR-08	Real-time collaboration — multiple users editing the same trip	✅ Implemented
FR-09	Social feed — publish trips, follow users, like/clone itineraries	✅ Implemented
FR-10	Multi-level undo/redo for itinerary edits	✅ Implemented
FR-11	Offline editing with automatic sync on reconnect	✅ Implemented

2.2 Non-Functional Requirements

ID	Requirement	Mechanism
NFR-01	Authentication on every API request	Firebase JWT verification middleware
NFR-02	API key secrecy — keys never exposed to client	Server-side proxying via Express gateway
NFR-03	Rate limiting to protect free-tier API quotas	`express-rate-limit`: 100 req/15 min general, 50 req/hr for external APIs
NFR-04	Response caching to reduce latency and API costs	NodeCache (30 min TTL for POIs), ETag middleware
NFR-05	Offline resilience	AsyncStorage-backed `localCache` + persistent `syncQueue`
NFR-06	Graceful degradation when external APIs fail	Fallback chain: Google Places → ORS → cached results
NFR-07	Security headers and CORS	Helmet.js, CORS middleware
NFR-08	Structured error handling	Global Express error handler, typed API errors in client

---

3. System Architecture

3.1 High-Level Overview

```
┌──────────────────────────────────────────────────────────┐
│                    React Native Client                   │
│   Expo Router │ Zustand Stores │ Axios API Client        │
│   localCache (AsyncStorage) │ syncQueue (AsyncStorage)   │
```

```
└──────────┬──────────────────────┬──────────────┘
           │ REST (HTTP/JWT)      │ Firebase SDK
           ▼                      ▼
┌──────────────────┐   ┌──────────────────────┐
│ Express API Gateway │   │ Firebase Realtime DB  │
│ auth · cache · limiter│   │ Real-time trip events │
└──────────┬──────────┘   └──────────────────────┘
           │
    ┌──────┴──────┐
    │ TravelService│ (Facade)
    │   Facade    │
    └──────┬──────┘
           │      │
┌──────▼──┐ ┌──▼──────┐   ┌──────────────┐
│ Google  │ │   ORS   │   │ MongoDB Atlas │
│ Places  │ │ Adapter │   │ Trip/User/Exp │
│ Adapter │ │(VROOM/POI)│   │ eense/FeedPost │
└────────┘ └─────────┘   └──────────────┘
                 │
       Nominatim (Geocoding fallback)
```

3.2 Backend Layer Structure

Layer	Location	Responsibility
Presentation	`backend/src/routes/`	Express route definitions, request parsing, response formatting
Middleware	`backend/src/middleware/`	Auth (JWT), caching (NodeCache), rate limiting
Business Logic	`backend/src/services/`	TravelService facade, API adapters
Data Access	`backend/src/models/`	Mongoose ODM schemas (Trip, User, Expense, FeedPost)
Infrastructure	`backend/src/config/`	MongoDB connection, Firebase Admin SDK init

3.3 Frontend Layer Structure

Layer	Location	Responsibility
View	`frontend/app/`	Expo Router screens — `(auth)`, `(tabs)`, `trip/`
State	`frontend/stores/`	Zustand stores: `tripStore`, `budgetStore`, `feedStore`, `authStore`, `activeTripStore`
Service	`frontend/services/`	`api.ts` (Axios + JWT interceptor), `syncQueue.ts` (offline queue + local cache)
Config	`frontend/config/`	Firebase JS SDK init, environment variables, dynamic API base URL detection

---

4. Design Decisions & ADRs

ADR-001: Firebase Realtime Database for Collaboration Events

Context: Multiple users can collaborate on a single trip simultaneously. The app required a mechanism to propagate changes to all connected clients in real time without polling.

Decision: Use Firebase Realtime Database (RTDB) as the event bus for collaboration. When a trip mutation is made (add stop, reorder, update), the backend writes the sanitised full trip document to the Firebase path `trips/{tripId}`. All connected clients have a listener (`onValue`) on this path and apply the received document directly via Last-Write-Wins (LWW).

Rationale:

- Firebase RTDB provides sub-100ms event delivery with no infrastructure to manage.
- Writing the full sanitised document (rather than a diff/patch) means clients never need a secondary MongoDB fetch on each event — they apply the payload directly.
- Avoids a WebSocket server that would need to be hosted and scaled.

Consequences (Positive): Instant synchronisation, no polling overhead, simple client-side subscription model.
Consequences (Negative): Larger Firebase write payloads for trips with many stops; Firebase RTDB billed on data transfer.

ADR-002: API Gateway Pattern — All External API Calls Proxied Through Backend

Context: Google Places, ORS, and Nominatim all require API keys. Embedding keys in the mobile app bundle would expose them to reverse engineering.

Decision: The frontend never calls external APIs directly. All POI searches, geocoding, and route optimisation requests go through the Express backend, which holds keys in server-side environment variables.

Rationale: Prevents API key theft, allows server-side caching to reduce quota consumption, and centralises rate-limiting logic.

Consequences (Positive): Keys are safe; caching reduces cost; single audit point for all external calls.
Consequences (Negative): Extra network hop for every POI/route request; backend becomes a dependency for features that could theoretically work client-side.

ADR-003: Zustand for Frontend State Management

Context: The app needed global state (trip list, current trip, budget, feed, auth) accessible across deeply nested screens without prop drilling.

Decision: Use Zustand over Redux or React Context. Each domain has its own store (`tripStore` , `budgetStore` , `feedStore` , `authStore` , `activeTripStore`).

Rationale: Zustand has zero boilerplate compared to Redux, is lighter than Context for frequent updates, and stores are plain objects with actions — easy to reason about and test.

Consequences (Positive): Minimal boilerplate, fast re-renders via selector subscriptions, easy to colocate async logic with state.
Consequences (Negative): No built-in DevTools (Redux has excellent time-travel debugging); state is not serialised out-of-the-box (addressed manually via `localCache`).

ADR-004: Offline-First with AsyncStorage Sync Queue

Context: Mobile users frequently lose connectivity. Write operations (create trip, add stop, log expense) should not fail silently when offline.

Decision: Implement a persistent `syncQueue` in AsyncStorage. Every store's write operation wraps its API call in a try-catch; on failure, the mutation is pushed to the queue with `{ method, url, data }`. The queue is flushed

automatically via an `AppState` listener in `_layout.tsx` each time the app becomes active.

Rationale: Gives users a seamless experience — they can keep editing offline and changes propagate when connectivity returns. No manual user action required.

Consequences (Positive): Resilient to network dropouts; automatic reconciliation; covers all five stores uniformly.
Consequences (Negative): Queued mutations could conflict if another user edits the same resource while offline; conflict resolution is LWW (last writer wins), which may occasionally lose data.

---

5. Design Patterns

5.1 Facade Pattern

Location: `backend/src/services/travelService.js`

`TravelService` exposes three clean methods (`searchPOI` , `searchByCategory` , `getOptimizedRoute` , `geocode`) to all route handlers. Internally it orchestrates `PlacesAdapter` , `ORSAdapter` , and the NodeCache — hiding all retry, fallback, distance calculation, and result-merging complexity.

```
Route Handler → travelService.searchPOI(query, lat, lng, radius)
                    ├── cache.get(key)                [cache hit → return]
                    ├── PlacesAdapter.searchPOI()     [primary]
                    ├── ORSAdapter.searchPOI()        [fallback if Places returns 0]
                    ├── _calculateDistance()          [Haversine sort]
                    └── cache.set(key, results, 1800) [cache for 30 min]
```

5.2 Adapter Pattern

Location: `backend/src/services/placesAdapter.js` , `orsAdapter.js` , `overpassAdapter.js`

Each adapter wraps a third-party API with a different contract and normalises its responses into the application's unified POI schema `{ name, lat, lng, category, rating, photo, address, placeId }` . The `TravelService` facade calls adapters interchangeably — swapping providers requires no changes outside the adapter.

5.3 Strategy Pattern

The Adapter + Facade combination realises the Strategy pattern: `TravelService` selects a POI strategy (Google Places or ORS) at runtime based on which adapter returns results. The selection logic is encapsulated inside the facade, transparent to callers.

5.4 Singleton Pattern

Instance	Location	Purpose
Firebase Admin SDK	`backend/src/config/firebase.js`	Single admin auth instance
Firebase JS SDK	`frontend/config/firebase.ts`	Single client auth + RTDB instance
NodeCache	`backend/src/middleware/cache.js`	Single in-process cache store
TravelService	`backend/src/services/travelService.js`	Shared across all routes
Axios client	`frontend/services/api.ts`	Single client with JWT interceptors

5.5 Command Pattern

Location: `frontend/stores/tripStore.ts`

Every itinerary mutation (`updateTrip` , `addStop` , `removeStop` , `reorderStops`) pushes a `Command` object onto `undoStack` before applying the change:

```
interface Command {
    type: 'ADD_STOP' | 'REMOVE_STOP' | 'REORDER_STOPS' | 'UPDATE_TRIP';
    tripId: string;
    dayIndex?: number;
    data: any;
    previousState: any; // snapshot for rollback
}
```

`undo()` pops from `undoStack` , reverses the API mutation, and pushes to `redoStack` . `redo()` replays. This gives users multi-level undo/redo without requiring the server to store history.

5.6 Observer Pattern

Location: `frontend/stores/tripStore.ts` — `subscribeTripUpdates()` , `frontend/stores/authStore.ts` — `initialize()`

Firebase RTDB's `onValue()` listener is a textbook Observer: the trip store subscribes to the path `trips/{tripId}` , and Firebase pushes a notification every time any collaborator writes a change. `onAuthStateChanged()` in `authStore` subscribes to Firebase Auth state, automatically routing users to the auth screen on sign-out.

5.7 Repository Pattern

Location: `backend/src/models/` + `backend/src/routes/`

Mongoose models serve as repositories. Route handlers never construct raw MongoDB queries — they delegate entirely to the model's built-in methods (`Trip.findById()` , `Trip.findByIdAndUpdate()` , `Expense.find()`). The data-access layer is cleanly separated from business logic.

5.8 Cache-Aside (Proxy) Pattern

Two levels of cache-aside are implemented:

1. **Server-side:** `travelService.searchPOI()` checks NodeCache before calling external APIs; results are stored on cache miss.
2. **Client-side:** All Zustand store `fetch*` methods check `localCache` (AsyncStorage) on API failure, serving stale data rather than showing an error.

---

6. Architectural Tactics

6.1 Performance

Tactic	Implementation
Server caching	NodeCache — 30 min TTL for POI/category results, keyed by `query:lat:lng:radius`
ETag response caching	`backend/src/middleware/cache.js` middleware on all GET endpoints

Client caching	`localCache` in AsyncStorage — trips, budget, feed persisted across sessions
Proximity sorting	Haversine distance calculation + top-15 result capping in `travelService`
Lazy screen loading	Expo Router loads screens on-demand via file-based routing
Pagination	Cursor-based pagination on feed endpoints

6.2 Security

Tactic	Implementation
Centralised authentication	`authenticate` middleware verifies Firebase JWT on every `/api/*` route
Authorization checks	Trip owner / collaborator membership validated in route handlers before mutations
API key secrecy	All third-party keys in `backend/.env`; client never sees them
HTTP security headers	`helmet()` applied globally in `index.js`
Rate limiting	100 req/15 min general; 50 req/hr for external API proxy routes
Input sanitisation	`trim: true` on Mongoose string fields; validated enums for categories/status

6.3 Availability

Tactic	Implementation
Graceful degradation	Google Places → ORS fallback in `travelService.searchPOI()`; Nominatim → Google for geocoding
Offline resilience	`syncQueue` queues writes; `localCache` serves reads when API is unreachable
Error boundaries	Try-catch in all store actions; user-facing error state exposed via Zustand
Health check endpoint	`GET /api/health` returns `{ status: 'ok', timestamp }` — unauthenticated

6.4 Modifiability

Tactic	Implementation
Information hiding	Each API adapter hides provider-specific request/response formats
Separation of concerns	Strict layering: routes → service → model; screens → store → api client
Dependency inversion	Route handlers depend on `TravelService` interface, not concrete adapters
Parameterisation	All environment-specific config in `.env` (backend) and `env.ts` (frontend)

---

7. Data Models

7.1 Trip (MongoDB)

```
Trip {
  name, destination, startDate, endDate, coverImage
  owner: String (Firebase UID)
  collaborators: [String]           // Firebase UIDs
  days: [Day]
    └── Day { date, dayNumber, stops: [Stop] }
          └── Stop { name, placeId, lat, lng, category,
                     notes, order, arrivalTime, duration,
                     cost, expenseId, address, rating, photo }
  isPublished, publishedAt, status: 'planning'|'active'|'completed'
  timestamps (createdAt, updatedAt)
}
```

Indexes: `{ owner, createdAt }`, `{ collaborators }`, `{ isPublished, publishedAt }`

7.2 User (MongoDB)

```
User {
  firebaseUid: String (unique)
  email, displayName, avatarUrl
  followers: [String]           // Firebase UIDs
  following: [String]
  followRequests: [String]      // pending incoming
  pendingFollowing: [String]    // pending outgoing
  publishedTrips: [ObjectId → Trip]
  timestamps
}
```

7.3 Expense (MongoDB)

```
Expense {
  trip: ObjectId → Trip
  user: String (Firebase UID)
  description, amount, currency
  category: 'accommodation'|'food'|'transport'|'activities'|'other'
  date, dayNumber
  splitAmong: [String]          // Firebase UIDs
  timestamps
}
```

7.4 FeedPost (MongoDB)

```
FeedPost {
  trip: ObjectId → Trip
  author: String (Firebase UID)
  caption, likes: [String]
```

```
  publishedAt, timestamps
}
```

---

8. Key Feature Implementations

8.1 Trip Itinerary Builder

The trip detail screen (`frontend/app/trip/[id].tsx`) renders a day-by-day itinerary. Each day is a collapsible section containing a draggable list of stops. The `tripStore` exposes:

- `addStop(tripId, dayIndex, stop)` — POST to `/api/trips/:id/days/:dayIndex/stops`
- `removeStop(tripId, dayIndex, stopId)` — DELETE equivalent
- `reorderStops(tripId, dayIndex, stopOrder)` — PUT with new order array
- `updateStop(tripId, dayIndex, stopId, stop)` — PUT for individual stop edits

Each mutation captures `previousState` and pushes a `Command` for undo/redo.

8.2 Route Optimisation

`POST /api/routes/optimize` receives an array of stops with coordinates. The `ORSAdapter` submits a VROOM (Vehicle Routing Optimisation with Multiple stops) job to the ORS API, which returns the optimal visit order plus a GeoJSON polyline. The frontend map (`frontend/app/trip/map/[id].tsx`) renders the polyline as an overlay on the Leaflet WebView map.

8.3 POI Search

`GET /api/poi/search?q=&lat=&lng=&radius=` is handled by `travelService.searchPOI()` . The flow:

1. Check NodeCache for `poi:{q}:{lat}:{lng}:{radius}`
2. Call `PlacesAdapter.searchPOI()` (Google Places Text Search)
3. If zero results, fall back to `ORSAdapter.searchPOI()` (ORS Pelias geocoder)
4. Compute Haversine distance for each result, sort ascending, cap at 15 results
5. Store in cache (30 min TTL) and return

The frontend `Explore` tab supports 13 categories (hotel, restaurant, landmark, activity, transport, shopping, museum, park, nightlife, medical, grocery, finance, other) with a radius slider.

8.4 Budget Tracker

`budgetStore` fetches expenses via `GET /api/budget/:tripId` , falls back to `localCache` on failure. Mutations (add/update/delete expense) are queued in `syncQueue` on network failure. The budget screen calculates:

- Total expenditure per category
- Per-person split based on `splitAmong` arrays
- Daily breakdown aligned with trip `dayNumber` fields

8.5 Social Feed

The feed screen renders published trips from followed users. `feedStore` exposes:

- `fetchFeed()` — paginated `GET /api/feed`
- `likePost(postId)` / `unlikePost(postId)` — `POST /api/feed/:id/like`
- `publishTrip(tripId, caption)` — creates a `FeedPost` document
- `cloneTrip(tripId)` — deep-copies an itinerary to the current user's trips

`likePost` and `publishTrip` both queue in `syncQueue` on failure.

8.6 Offline Sync Queue

```
// syncQueue lifecycle
syncQueue.add({ method: 'POST', url: '/trips', data })   // on write failure
syncQueue.flush()                                          // on app become active
syncQueue.clear()                                          // on sign-out
```

The queue sorts by `timestamp` before flushing to preserve operation ordering. Failed flushes leave actions in the queue for the next attempt. The `_layout.tsx` root layout subscribes to `AppState` changes and triggers `flush()` whenever the app returns from background — no manual user action required.

---

9. External API Integration

9.1 Google Places API

- **Usage:** Primary POI search (text query + location bias), place details (rating, photo, address)
- **Endpoint used:** Places Text Search, Place Details
- **Key location:** `backend/.env → GOOGLE_PLACES_API_KEY`
- **Rate protection:** `externalApiLimiter` (50 req/hr), server-side cache (30 min TTL)

9.2 OpenRouteService (ORS)

- **Usage:** Fallback POI search (Pelias geocoder), route optimisation (VROOM solver), driving directions
- **Endpoint used:** Geocode Search, VROOM optimisation, Directions (GeoJSON)
- **Key location:** `backend/.env → ORS_API_KEY`

9.3 Nominatim (OpenStreetMap)

- **Usage:** Primary geocoding (city/address → coordinates) for trip destination lookup
- **Endpoint:** `https://nominatim.openstreetmap.org/search`
- **No key required**, but `User-Agent: WanderMate/1.0` header sent per OSM policy
- Falls back to Google Places Text Search if result is empty or request times out

9.4 Firebase Realtime Database

- **Usage:** Real-time trip update events for collaborative editing
- **Data paths:** `trips/{tripId}` — full sanitised trip document written on every mutation
- **SDK:** Firebase Admin SDK (server) + Firebase JS SDK (client)

9.5 API Ninjas (Trip Card Images)

- **Usage:** Random travel background images for trip cards
- **Endpoint:** `GET https://api.api-ninjas.com/v1/randomimage` — returns binary JPG
- **Key location:** `frontend/config/env.ts → API_NINJAS_KEY`
- **Caching:** Cached in AsyncStorage as base64 data URI under `@wandermate_trip_img_{tripId}`, fetched once per trip

---

10. Security Implementation

10.1 Authentication Flow

```
Client                    Backend                  Firebase Auth
  |                          |                          |
  |── POST /api/auth ───────X  (no such route)          |
  |                                                     |
  |  signInWithEmailAndPassword(email, password) ──────►|
  |◄─── Firebase ID Token ──────────────────────────────|
  |                                                     |
  |── GET /api/trips  ──────►|                          |
  |   Authorization: Bearer  |                          |
  |   {Firebase ID Token}    |── verifyIdToken() ──────►|
  |                          |◄── { uid, email, ... } ──|
  |                          |  req.user = decoded       |
  |◄── 200 { trips: [...] } ─|                          |
```

Every route is protected by the `authenticate` middleware in `backend/src/middleware/auth.js`. The middleware extracts the Bearer token, verifies it against Firebase Admin SDK, and attaches the decoded user object to `req.user`. Downstream handlers use `req.user.uid` for ownership checks.

10.2 Authorization

Trip mutations check that `req.user.uid === trip.owner || trip.collaborators.includes(req.user.uid)` before allowing changes. Delete operations are restricted to the owner only.

10.3 Rate Limiting

Two tiers:

- **General:** `rateLimit({ windowMs: 15 * 60 * 1000, max: 100 })` — all `/api/*` routes
- **External API proxy:** `rateLimit({ windowMs: 60 * 60 * 1000, max: 50 })` — POI and route endpoints

This prevents quota exhaustion on paid APIs and protects against brute-force attempts.

10.4 Secret Management

No API keys appear in the frontend bundle. The client sends requests to the backend gateway, which holds all third-party credentials in `backend/.env` (server-side only, excluded from version control via `.gitignore`).

---

11. Offline-First Architecture

The offline-first design has two components:

11.1 Local Cache (Read Path)

`localCache` in `frontend/services/syncQueue.ts` wraps AsyncStorage with a key-value store that records `{ data, cachedAt }`. Every `fetch*` function in the Zustand stores follows the pattern:

```
try {
    const response = await api.get('/trips');
    set({ trips: response.data });
    await localCache.set('trips', response.data);   // update cache on success
} catch {
    const cached = await localCache.get<Trip[]>('trips', Infinity); // serve stale on
```

```
failure
    if (cached) set({ trips: cached });
}
```

`Infinity` as `maxAgeMs` means stale data is always preferred over an empty screen.

11.2 Sync Queue (Write Path)

Every write operation in all five Zustand stores follows the pattern:

```
try {
    await api.post('/trips', data);
} catch {
    await syncQueue.add({ method: 'POST', url: '/trips', data }); // persist for later
}
```

The queue is an array of `QueuedAction` objects persisted to AsyncStorage under `@wandermate_sync_queue`. On flush, actions are replayed in timestamp order. Failed actions remain in the queue. The `AppState` listener in `_layout.tsx` flushes on every `'active'` transition (app foreground), ensuring sync happens automatically without user intervention.

11.3 Coverage

All five stores implement offline write support:

Store	Queued Operations
`tripStore`	createTrip, updateTrip, deleteTrip, addStop, updateStop, removeStop, reorderStops, addCollaborator
`budgetStore`	addExpense, updateExpense, deleteExpense
`feedStore`	likePost, publishTrip
`authStore`	followUser, unfollowUser, acceptFollowRequest, denyFollowRequest

---

12. Real-Time Collaboration

12.1 Mechanism

When a user saves a trip mutation, the backend route handler:

1. Writes the change to MongoDB
2. Writes the full sanitised trip document to Firebase RTDB at `trips/{tripId}`

All connected collaborators have a `subscribeTripUpdates(tripId)` listener active on that path. When Firebase delivers the event, the store applies the document directly to `currentTrip` in Zustand state, immediately reflecting the change in the UI.

12.2 Last-Write-Wins

Concurrent edits are resolved by LWW semantics: the last write to Firebase RTDB becomes the authoritative state. This is simple and sufficient for travel itineraries where simultaneous edits to the same stop are unlikely. A future

improvement could add optimistic locking via version vectors.

12.3 Listener Lifecycle

`subscribeTripUpdates()` returns an unsubscribe function. The trip detail screen calls it in a `useEffect` cleanup to prevent memory leaks and Firebase listener accumulation when navigating away.

---

13. Frontend State Management

13.1 Store Architecture

Five Zustand stores handle distinct domains:

Store	Domain	Key State
`tripStore`	Trip CRUD, itinerary, undo/redo	`trips[]`, `currentTrip`, `undoStack`, `redoStack`
`budgetStore`	Expense tracking	`expenses[]`, `total`, `byCategory`
`feedStore`	Social feed	`posts[]`, `likedPosts`
`authStore`	Firebase auth, social graph	`firebaseUser`, `profile`, `isAuthenticated`
`activeTripStore`	Active trip for quick-access widget	`activeTripId`

13.2 Navigation & Routing

Expo Router provides file-based navigation. The route tree:

```
app/
├── _layout.tsx          Root layout (auth gate, sync flush, theme)
├── (auth)/
│   ├── _layout.tsx
│   └── sign-in.tsx      Email/password auth
├── (tabs)/
│   ├── _layout.tsx      Bottom tab navigator
│   ├── index.tsx        My Trips
│   ├── explore.tsx      POI Search
│   ├── feed.tsx         Social Feed
│   ├── budget.tsx       Budget Tracker
│   └── profile.tsx      User Profile + Sync controls
└── trip/
    ├── [id].tsx         Trip Detail (itinerary editor)
    ├── create.tsx       Trip Creation wizard
    └── map/[id].tsx     Interactive Leaflet Map
```

The `AuthGate` component in `_layout.tsx` redirects unauthenticated users to `/(auth)/sign-in` and authenticated users away from auth screens.

13.3 Dynamic API Base URL

`frontend/config/env.ts` implements a detection strategy for the API base URL:

1. Reads `Constants.expoConfig.hostUri` from the Expo runtime

2. If running on a physical device (Expo Go), extracts the development server's LAN IP
3. Constructs `http://{host}:5000/api` dynamically
4. Falls back to `http://10.0.2.2:5000/api` (Android emulator) or `http://127.0.0.1:5000/api` (iOS simulator)

This eliminates the need to hardcode the developer's machine IP when testing on physical devices.

---

14. Testing & Quality Assurance

14.1 Manual Testing Approach

Testing was conducted through iterative manual testing on:

- Android physical device via Expo Go
- Android emulator (AVD)
- Web browser (Expo web build)

Each feature was tested end-to-end: creation flow → edit → offline simulation (airplane mode) → reconnect → verify sync.

14.2 TypeScript

The frontend is fully typed with TypeScript. Core domain types (`Trip` , `Stop` , `Day` , `User` , `Expense`) are defined in `frontend/types/index.ts` and used consistently across stores, screens, and service layer. The `tsconfig.json` enforces strict mode.

14.3 ESLint

`eslint.config.js` uses Expo's default ESLint config with React hooks rules. No custom overrides beyond the Expo preset.

14.4 Error Handling

All Zustand store actions expose an `error` state field. Screens observe `error` via selector and display user-friendly messages. The global Express error handler in `index.js` returns structured JSON errors; the Axios response interceptor in `api.ts` normalises them into plain `Error` objects for uniform handling in stores.

---

15. Challenges & Lessons Learned

15.1 Dynamic LAN IP for Physical Device Testing

Challenge: When testing on a physical Android device via Expo Go, `http://localhost:5000` does not reach the development machine. Hardcoding the LAN IP broke other developers' setups and changed every session.

Solution: Implemented dynamic host detection in `env.ts` by reading `Constants.expoConfig.hostUri` . This extracts the LAN IP Expo is already broadcasting, making the API URL auto-resolve for any developer on any machine without manual edits.

Lesson: Never hardcode network addresses in a mobile app. Expose host detection as a utility and derive all URLs from it.

15.2 Firebase RTDB Write Strategy — Patch vs. Full Document

Challenge: Initial design wrote only a small `TRIP_UPDATED` event (just changed fields) to Firebase. This meant clients had to make a secondary MongoDB fetch to get the current state, adding latency and a potential race condition where the client's fetch arrived before the MongoDB write committed.

Solution: Changed the Firebase write to push the full sanitised trip document. Clients apply it directly without a round-trip. LWW semantics hold naturally.

Lesson: For real-time collaboration with eventual consistency, shipping full documents is simpler and more reliable than incremental patches unless the document is extremely large.

15.3 External API Quota Management

Challenge: Google Places API has strict quota limits on the free tier. During development, frequent POI searches exhausted the daily quota, breaking the feature for the rest of the day.

Solution: Implemented 30-minute server-side NodeCache for all POI results, plus the ORS fallback so search still works when Google is unavailable. Added the `externalApiLimiter` middleware (50 req/hr) as a safety valve.

Lesson: Always implement caching and fallback strategies before integrating quota-limited external APIs, not after hitting the quota wall.

15.4 Offline Sync Queue Ordering

Challenge: When the sync queue flushed after reconnect, operations applied in insertion order could be incorrect if the user had performed dependent operations offline (e.g., add stop then immediately remove it — the remove references an `_id` that hasn't been created server-side yet).

Solution: Sort queue by `timestamp` before flushing to maintain chronological order. For dependent operations (add then remove), the add's response `_id` must be captured. This edge case is a known limitation — the queue does not currently resolve cross-operation ID dependencies.

Lesson: Offline queue design needs to account for ID dependencies between operations. A future improvement would be to use local UUIDs as optimistic IDs and resolve them against server responses.

15.5 Leaflet Maps in React Native WebView

Challenge: React Native has no native Leaflet binding. Rendering an interactive map with markers, polylines, and bidirectional communication required embedding Leaflet in a WebView with a JavaScript bridge.

Solution: The map screen injects an HTML page containing Leaflet.js into a `WebView`. React Native communicates with the map via `webViewRef.current.injectJavaScript()` and receives events via `onMessage`. This allows adding/removing markers and rendering route polylines from native code.

Lesson: WebView bridges are powerful but fragile — serialisation errors and timing issues are common. Thorough message schema definition and defensive parsing on both sides are essential.

15.6 Google Sign-In on Mobile

Challenge: Google OAuth worked on the web browser but failed on the physical Android device. The root causes were: scheme mismatch between `app.json` (`"frontend"`) and `makeRedirectUri({ scheme: 'wandermate' })` ; no Android OAuth client ID registered in Google Cloud Console; and Expo Go generating `exp://` redirect URIs which Google rejects.

Resolution: The complexity of configuring native Google OAuth in Expo managed workflow (requiring a custom development build, SHA-1 fingerprint registration, and correct scheme alignment) led to the decision to remove Google Sign-In from the mobile app entirely. Email/password authentication is used exclusively, which works identically on all platforms without platform-specific OAuth configuration.

Lesson: Native OAuth flows in Expo managed workflow require significantly more setup than web OAuth. For a prototype, email/password auth is more reliable across all platforms. Google Sign-In should be deferred to a custom development build where the full native module stack can be properly configured.

---

16. Conclusion

WanderMate demonstrates a production-quality architecture for a collaborative mobile application with real-time sync, offline resilience, and multi-service API integration. Key architectural achievements include:

- **Facade + Strategy** pattern cleanly abstracting multiple POI/routing providers with automatic fallback
- **Offline-first** design covering all 18 write operations across five Zustand stores, with automatic sync on reconnect
- **Firebase RTDB** as a lightweight event bus for real-time collaboration without a self-hosted WebSocket server
- **Command pattern** enabling multi-level undo/redo for itinerary edits at zero server cost
- **Security layering** — JWT authentication, ownership-based authorisation, rate limiting, and server-side key proxying — addressing OWASP Top 10 concerns

The project process reinforced that architectural patterns are most valuable when they solve real, recurring problems in a codebase — not as exercises. The Facade pattern removed significant duplication across six route handlers; the offline sync queue resolved a genuine user experience problem on mobile networks. The most difficult challenge — native Google OAuth in Expo managed workflow — ultimately taught that simplicity and cross-platform reliability should take priority over feature completeness in a prototype.

WanderMate — Plan trips together, explore the world.

17. Contributions

- **Arnav Deshmukh**- Task 1 and Task 4
- **Ayush Roy**- Task 1 and Task 4
- **Sanyam Agarwal**- Task 1 and Task 4
- **Raghav Grover**- Task 1 , Task 2 and Task 3
- **Karthik Malavathula** - Task 1 , Task 2 and Task 3